# STAT293.draft

Nancy Lopez

2025-11-29

## Step 1: Packages

## Step 2: Simulation Paramaters

```
set.seed(23434)

# Dimensions
K <- 20      # Number of individuals
T_len <- 30  # Total number of time points
d <- 20      # Number of variables (Dimension)

# Model Structure
p <- 2            # VAR lag order
sparsity <- 0.05  # Sparsity level (proportion of non-zero coefficients)
heterogeneity <- "medium"  # Level of heterogeneity ("low", "medium", "high")

# Data Generation & Noise
sigma_ep <- 0.1  # Standard deviation of noise
phi_max <- 0.9   # Upper bound for coefficient generation
phi_min <- 0.1   # Lower bound for coefficient generation

# Evaluation Settings
n_holdout  <- 5  # Number of time points withheld for forecasting (holdout)


# Code below (do not modify)
# Heterogeneity
frac_common <- switch(heterogeneity,
                      "low"    = 2/3,
                      "medium" = 1/2,
                      "high"   = 1/3)

# total number of nonzero elements in each d × d transition matrix
n_nonzero_total <- max(1, round(sparsity * d * d))

# number of nonzero elements for common effects
n_common <- max(1, round(frac_common * n_nonzero_total))

# number of nonzero elements for unique (individual-specific) effects
n_unique <- n_nonzero_total - n_common
```

## Step 3: Helper Function

### Companion Matrix (F)

$$\mathbf{F} = \begin{bmatrix} \mathbf{\Phi}_1 & \mathbf{\Phi}_2 & \cdots & \mathbf{\Phi}_{p-1} & \mathbf{\Phi}_p \\ \mathbf{I}_d & \mathbf{0} & \cdots & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_d & \cdots & \mathbf{0} & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & \mathbf{I}_d & \mathbf{0} \end{bmatrix}$$

```r
# Generate Coefficient Matrix (Can be used for common or unique)
gen_coef_matirx <- function(d, idx, min_val = phi_min, max_val = phi_max){

  M <- matrix(0, d, d)
  n_nonzero <- length(idx)
  M[idx] <- runif(n_nonzero, min_val, max_val)
  # M[idx] <- runif(n_nonzero, min_val, max_val) * sample(c(-1,1), n_nonzero, replace = TRUE)


  return(M)
}


# Determine Stability
check_phi_stability <- function(phi_array) {
  # Require input dimensions to be [row, col, lag]
  # Ensure input is a 3D array [d, d, p].
  if (!is.array(phi_array)) {
    phi_array <- array(phi_array, dim = c(d, d, 1))
  }

  d <- dim(phi_array)[1]
  p <- dim(phi_array)[3]

  # Convert 3D Array slices into a List
  phi_list <- lapply(1:p, function(i) phi_array[,,i])

  # --- the Companion Matrix (F) ---
  # The structure is a (p*d) x (p*d) matrix:
  # F = [ Phi1  Phi2 ... Phip ]
  #     [  I     0   ...   0  ]
  #     [  0     I   ...   0  ]
  #     ...

  # 1. Construct Companion Matrix (F)
  # Create Top Block: [Phi1, Phi2, ..., Phip]
  top_block <- do.call(cbind, phi_list)

  if (p > 1) {
    # Identity matrix on the sub-diagonal
    ident_block <- diag(1, d * (p - 1))
    # Zero column on the right
    zero_col <- matrix(0, nrow = d * (p - 1), ncol = d)
    bottom_block <- cbind(ident_block, zero_col)
```

```
    # Combine
    companion_matrix <- rbind(top_block, bottom_block)
  } else {
    # VAR(1) case: Companion matrix is just Phi1 itself
    companion_matrix <- top_block
  }

  # 2. Calculate Eigenvalues & Determine Stability
  eigen_values <- eigen(companion_matrix)$values
  max_eigen <- max(Mod(eigen_values))

  return(list(
    is_stable = max_eigen < 1,
    max_eigen = max_eigen
  ))
}
```

## Step 4: Generate Individual Matrix (pd)*d

### Generate Common Coefficients Matrix and Indices

```
# Position Indices
# Dimensions: [Number of Unique Elements, Subject, Lag]
# This stores the specific grid locations for each subject's unique effects
unique_indices_array <- array(0, dim = c(n_unique, K, p))

# Common Coefficients Matrix (mu)
# Dimensions: [Row, Col, Lag]
# This stores the shared coefficient matrices for the population
common_mu_array <- array(0, dim = c(d, d, p))

# Individual Matrix (B)
# Dimensions: [Row, Col, Lag, Subject]
# This will store the final stable transition matrices for every subject
individual_B <- array(0, dim = c(d, d, p, K))


for (lag in 1:p) {

  # 1. Generate Common Coefficients Matrix
  # Randomly select grid points for the common structure
  common_idx <- sample(1:(d*d), n_common)
  common_mu_array[,,lag] <- gen_coef_matirx(d, idx = common_idx)

  # 2. Allocate Unique Locations
  available_pool <- setdiff(1:(d*d), common_idx)

  # Sample indices for all K subjects at once from the remaining pool
  unique_idx <- sample(available_pool, n_unique * K, replace = FALSE)

  # Reshape into a matrix where each column represents one subject
  unique_indices_array[,,lag] <- matrix(unique_idx, nrow = n_unique, ncol = K)
```

```
}
```

### Generation Individual Matrix and Stability Check

```
for (k in 1:K) {

  is_stable <- FALSE
  attempt <- 0
  max_attempts <- 100

  while(!is_stable) {

    attempt <- attempt + 1

    # Temporary container for this subject's p matrices
    current_subject_phi <- array(0, dim = c(d, d, p))

    for (lag in 1:p) {
      # Retrieve Common Matrix (mu)
      common_mu <- common_mu_array[,,lag]

      # Looks up the specific locations pre-allocated for subject k at this lag
      unique_idx <- unique_indices_array[, k, lag]

      # Generate Unique Delta Matrix
      unique_Delta <- gen_coef_matirx(d, idx = unique_idx)

      # Phi Matrix at this lag
      current_subject_phi[,,lag] <- common_mu + unique_Delta
    }

    # Stability Check
    check_result <- check_phi_stability(current_subject_phi)

    if (check_result$is_stable) {
      # Success
      is_stable <- TRUE
      individual_B[,,,k] <- current_subject_phi

    } else {
      # Failure: Repeat 'while' loop and regenerate unique_Delta
      # Note: 'unique_idx' and 'common_mu' remain fixed.

      # Fallback mechanism if stability is hard to achieve
      if (attempt > max_attempts) {

         # Calculate Scaling Factor
         # The current max eigenvalue is > 1 (unstable).
         # Formula: scaling_factor = current_eigen / target_radius
         target_radius <- 0.95
         scaling_factor <- check_result$max_eigen / target_radius
```

```
        # Apply Scaling
        current_subject_phi <- current_subject_phi / scaling_factor

        # Force Stability Flag
        is_stable <- TRUE
        individual_B[,,,k] <- current_subject_phi

        warning(sprintf("Subject %d unstable after %d attempts. Forced scaling applied (Factor=%.2f)."
      }
    }
  }
}
```

```
## Warning: Subject 16 unstable after 100 attempts. Forced scaling applied
## (Factor=1.28).
```

## Step 5: Generate Var(p) ILD for each individual

```
gen_VAR_p <- function(phi_array, T_len, p, d, sigma_ep, burn_in = 50) {

  # Require input dimensions to be [row, col, lag]
  # Ensure input is a 3D array [d, d, p].
  if (!is.array(phi_array)) {
    phi_array <- array(phi_array, dim = c(d, d, 1))
  }
  p <- dim(phi_array)[3]

  # Total Simulation Length (Desired Length + Burn-in)
  total_T <- T_len + burn_in

  # Initialize
  X <- matrix(0, nrow = total_T, ncol = d)

  # Initialize the first 'p' steps to start the process
  X[1:p, ] <- matrix(rnorm(p * d), nrow = p, ncol = d)

  # Generate Data Iteratively
  for (t in (p + 1):total_T) {

    # Calculate the Autoregressive component: Sum(Phi_p * X_{t-p})
    ar_term <- numeric(d) # Initialize zero vector

    for (lag in 1:p) {
      # Retrieve coefficient matrix for this lag
      phi_p <- phi_array[,,lag]

      # Retrieve past data at t-lag
      past_X <- X[t - lag, ]

      # Matrix multiplication and accumulation
      ar_term <- ar_term + (phi_p %*% past_X)
    }
```

```
    # Add random error term
    X[t, ] <- ar_term + rnorm(d, 0, sigma_ep)
  }

  # Remove Burn-in Period
  # Return only the valid data sequence of length T_len
  return(X[(burn_in + 1):total_T, ])
}


# Generate Data for All Subjects
# data_list[[k]] contains the observed data matrix for subject k
# individual_B is 4D array [d, d, p, K]

data_list <- lapply(1:K, function(k) {

  # Get coefficients for subject k
  # Dimensions: [d, d, p]
  subject_phi <- array(individual_B[,,,k], dim = c(d, d, p))

  # Generate time series data
  # Returns a matrix of size [T_len, d]
  gen_VAR_p(subject_phi, T_len, p, d, sigma_ep)
})
```

## Step 6: Split Training & Test Datasets

```
# Define the split index (The end of the training period)
split_idx <- T_len - n_holdout

# Create Training Data
train_data <- lapply(data_list, function(X) { X[1:split_idx, ] })

# Create Test Data (With Lookback)
# Include the last 'p' rows of the training set.
# The "initial conditions (lags)" for the first prediction in the test set.
test_data <- lapply(data_list, function(X) { X[(split_idx + 1 - p):T_len, ] })

# ---  Note for Evaluation ---
# 'test_data' now has (n_holdout + p) rows. The first 'p' rows are history.
# Evaluate metrics only on the last 'n_holdout' rows (the actual test set).
```

## Step 7: (a) Individual LASSO for VAR(p)

### Functions

```
# Construct the VAR(p) Design Matrix
# ------
# Inputs:
#   X: The raw time series matrix [T x d]
#   p: Number of lags
```

```r
# ------
gen_design_mat <- function(X, p = 1) {
  t <- nrow(X) # Total time points (T)

  # Create Target Matrix Y (Current time t)
  Y_mat <- X[(p + 1):t, ]

  # Create Predictor Matrix X_lags (Past times)
  # Stack [ X_{t-1} | X_{t-2} | ... | X_{t-p} ] horizontally.
  lag_list <- vector("list", p)

  for (lag in 1:p) {
    # If Y ranges from [p+1] to [t],
    # then X_{t-lag} ranges from [p+1-lag] to [t-lag].
    lag_list[[lag]] <- X[(p + 1 - lag):(t - lag), ]
  }

  X_lags <- do.call(cbind, lag_list)

  Z_mat <- cbind(Y_mat, X_lags)

  return(Z_mat)
}


# Extract Estimated Coefficients from LASSO Results
# ------
# Output Dimension: [d] x [d * p]
# Structure: [ Phi_1 | Phi_2 | ... | Phi_p ] (Horizontally stacked)
# ------

extract_B_hat <- function(fit_list, d, p) {

  # Initialize an empty matrix to hold coefficients
  # Rows = Equations (Variables)
  # Cols = All predictors across all lags
  B_hat <- matrix(0, nrow = d, ncol = d * p)

  # Iterate through each variable j (each equation)
  for (j in 1:d) {

    model <- fit_list[[j]]

    # Extract coefficients at the optimal lambda "lambda.min"
    raw_coefs <- coef(model, s = "lambda.min")

    # Remove Intercept
    beta_vec <- as.vector(raw_coefs)[-1]

    # Safety Check: Ensure the length matches our expectation
    if (length(beta_vec) == d * p) {
      B_hat[j, ] <- beta_vec
    } else {
```

```
      warning(paste("Coefficient length mismatch for variable", j))
    }
  }

  return(B_hat)
}


# Reshape Flat Matrix to 3D Array
# True B Array [d, d, p].

reshape_to_array <- function(B_flat, d, p) {
  B_array <- array(0, dim = c(d, d, p))

  for (lag in 1:p) {
    # Calculate column indices for this specific lag in the flat matrix
    col_idx <- ((lag - 1) * d + 1) : (lag * d)

    # Slice the columns and store them in the 3rd dimension of the array
    B_array[,,lag] <- B_flat[, col_idx]
  }

  return(B_array)
}


# Fit Lasso for Individual VAR(p)
fit_lasso_var_p <- function(X, p = 1, lambda_ = NULL) {

  d <- ncol(X)

  Z_mat <- gen_design_mat(X, p)

  # The first d columns are Y_t (Current)
  Y_mat <- Z_mat[, 1:d]

  # The remaining columns are the stacked lags (Past)
  X_lags <- Z_mat[, -c(1:d)]

  # Fit LASSO for each variable j
  fit_list <- vector("list", d)

  for (j in 1:d) {
    fit_list[[j]] <- cv.glmnet(
      x = X_lags,
      y = Y_mat[, j],
      alpha = 1,       # alpha=1 specifies the LASSO penalty (L1 norm)
      lambda = lambda_, # If NULL, glmnet selects its own sequence
      nfolds = 5
    )
  }

  est_B_list <- extract_B_hat(fit_list, d, p)
```

```
  est_B_array <- reshape_to_array(est_B_list, d, p)

  return(est_B_array)
}
```

### Fit Lasso VAR(1) and VAR(p) for each subject

```
# fit LASSO for each person in train_data

# VAR(p)
if (p != 1){
  est_lasso_B_p <- lapply(train_data, function(data) {
  fit_lasso_var_p(data, p)
  })

  est_lasso_B_p <- array(unlist(est_lasso_B_p), dim = c(d, d, p, K))
}

# VAR(1)
est_lasso_B <- lapply(train_data, function(data) {
  fit_lasso_var_p(data, p = 1)
})
```

```
## Warning: from glmnet C++ code (error code -89); Convergence for 89th lambda
## value not reached after maxit=100000 iterations; solutions for larger lambdas
## returned
```

```
est_lasso_B <- array(unlist(est_lasso_B), dim = c(d, d, K))
```

## Step 8: (b) Multi-VAR

```
multivar_B <- function(est_multivar_B, d, K){
  est_multivar_B <- array(unlist(est_multivar_B), dim = c(d, d, K))
  return(est_multivar_B)
}
```

### LASSO

```
lambda_standard <- 10^seq(log10(0.01), log10(5), length.out = 30)

multivar_lasso_model <- constructModel(
  data = train_data,
  lag = 1,
  #lambda1 = lambda_standard,
  #lambda2 = lambda_standard,
  standardize = TRUE,
  canonical = FALSE,
  lassotype = "standard", # standard multi-VAR (not adaptive)
  cv = "blocked",
  nfolds = 10
)
```

```
multivar_lasso_res <- cv.multivar(multivar_lasso_model)
```

```
##   |                                                                      |
```

```
est_multivar_lasso_B <- multivar_B(multivar_lasso_res$mats$total, d, K)
```

### Adaptive

```
lambda_adaptive <- 10^seq(log10(0.01), log10(5), length.out = 30)

multivar_adapt_model <- constructModel(
  data = train_data,
  lag = 1,
  #lambda1 = lambda_adaptive,
  #lambda2 = lambda_adaptive,
  standardize = TRUE,
  #weightest = "ridge",
  canonical = FALSE,
  lassotype = "adaptive",
  cv = "blocked",
  nfolds = 10
)

multivar_adapt_res <- cv.multivar(multivar_adapt_model)
```

```
##   |                                                                      |
```

```
est_multivar_adapt_B <- multivar_B(multivar_adapt_res$mats$total, d, K)
```

## Step 9: Evaluation

### Error

```
B_true <- individual_B[ , , 1, ]

diff_func <- function(est_B, B_true){
  sum((est_B - B_true)^2)
}

diffB_lasso <- diff_func(est_lasso_B, B_true)
diffB_multivar_lasso <- diff_func(est_multivar_lasso_B, B_true)
diffB_multivar_adapt <- diff_func(est_multivar_adapt_B, B_true)

diff_table <- data.frame(
  Method = c("LASSO (Lag 1)", "Multi-VAR (Standard)", "Multi-VAR (Adaptive)"),
  Error_Metric = c(diffB_lasso, diffB_multivar_lasso, diffB_multivar_adapt),
  stringsAsFactors = FALSE
)

if (p != 1) {
  diffB_lasso_p <- diff_func(est_lasso_B_p, individual_B)

  extra_row <- data.frame(
```

```
    Method = paste0("LASSO (Lag ", p, ")"),
    Error_Metric = diffB_lasso_p,
    stringsAsFactors = FALSE
  )

  diff_table <- rbind(diff_table, extra_row)
}

diff_table
```

```
##                 Method Error_Metric
## 1        LASSO (Lag 1)    123.41789
## 2 Multi-VAR (Standard)     50.69995
## 3 Multi-VAR (Adaptive)     75.94155
## 4        LASSO (Lag 2)    144.77467
```

```
kable(diff_table, digits = 3,
      caption = "Comparison of Estimation Error")
```

Table 1: Comparison of Estimation Error

| Method | Error_Metric |
|---|---|
| LASSO (Lag 1) | 123.418 |
| Multi-VAR (Standard) | 50.700 |
| Multi-VAR (Adaptive) | 75.942 |
| LASSO (Lag 2) | 144.775 |

## Sensitivity and Specificity

```
sens_spec_func <- function(B_true, B_est, thr = 1e-4) {
  true_nonzero <- abs(B_true) != 0
  est_nonzero <- abs(B_est) > thr

  TP <- sum(est_nonzero & true_nonzero)
  FN <- sum(!est_nonzero & true_nonzero)
  TN <- sum(!est_nonzero & !true_nonzero)
  FP <- sum(est_nonzero & !true_nonzero)

  if ((TP + FN) > 0) {
    sensitivity <- TP / (TP + FN)
  } else {
    sensitivity <- NA
  }

  if ((TN + FP) > 0) {
    specificity <- TN / (TN + FP)
  } else {
    specificity <- NA
  }

  return(list(
    sensitivity = sensitivity,
    specificity = specificity,
```

```
    TP = TP, FP = FP, TN = TN, FN = FN
  ))

}


# Calculate Mean Metrics Across All Subjects
# ------
# Inputs:
#   B_true : 3/4 Array [d, d, (p), K] containing true values
#   B_est  : 3/4D Array [d, d, (p), K] containing estimates
#   thr       : Threshold (default 1e-4)
#   output_type
# ------
calc_metrics <- function(B_true, B_est, thr = 1e-4,
                              output_type = c("mean", "subject", "both")) {

  # output_type = "mean" (default)
  output_type <- match.arg(output_type)

  # Check dimensions
  if (!all(dim(B_true) == dim(B_est))) {
    stop("Error: Dimensions of True and Estimated arrays do not match!")
  }

  dims <- dim(B_true)
  ndim <- length(dims)

  if (ndim == 3) {
    # CASE A: Input is 3D [d, d, K] (Implies VAR(1), p=1)
    d <- dims[1]
    K <- dims[3]

    # "Reshape" to 4D by adding a singleton dimension for p=1
    dim(B_true) <- c(d, d, 1, K)
    dim(B_est)  <- c(d, d, 1, K)

  } else if (ndim == 4) {
    K <- dims[4]

  } else {
    stop("Error: Input arrays must be 3D [d,d,K] or 4D [d,d,p,K]")
  }

  sens_vec <- numeric(K)
  spec_vec <- numeric(K)

  # Loop through each subject
  for (k in 1:K) {
    # Slice the 4D array to get the 3D block [d, d, p] for subject k
    # This block contains all lags for this person
    B_true_k <- B_true[, , , k]
    B_est_k  <- B_est[, , , k]
```

```
    # Calculate metrics for this person
    res <- sens_spec_func(B_true_k, B_est_k, thr = thr)

    sens_vec[k] <- res$sensitivity
    spec_vec[k] <- res$specificity
  }

  # Return the Mean across all subjects
  if (output_type == "mean") {

      return(list(
        mean_sens = mean(sens_vec),
        mean_spec = mean(spec_vec)
      ))

    } else if (output_type == "subject") {

      return(list(
        sensitivity = sens_vec,
        specificity = spec_vec
      ))

    } else { # "both"

      return(list(
        summary = list(mean_sens = mean(sens_vec), mean_spec = mean(spec_vec)),
        raw_data = list(sens_vec = sens_vec, spec_vec = spec_vec)
      ))
    }
}
```

```
# LASSO
metrics_lasso <- calc_metrics(
  B_true,
  B_est = est_lasso_B,
  output_type = "mean"
  )

# multivar lasso
metrics_multivar_lasso <- calc_metrics(
  B_true,
  B_est = est_multivar_lasso_B,
  output_type = "mean"
  )

# multivar adaptive
metrics_multivar_adapt <- calc_metrics(
  B_true,
  B_est = est_multivar_adapt_B,
  output_type = "mean"
  )
```

```r
# results
metrics_var1 <- data.frame(
  Method= c("LASSO (Lag 1)", "Multi-VAR (Standard)", "Multi-VAR (Adaptive)"),
  Sensitivity = c(metrics_lasso$mean_sens,
                  metrics_multivar_lasso$mean_sens,
                  metrics_multivar_adapt$mean_sens),
  Specificity = c(metrics_lasso$mean_spec,
                  metrics_multivar_lasso$mean_spec,
                  metrics_multivar_adapt$mean_spec)
  )

ss_dat <- metrics_var1

if (p != 1) {
  metrics_lasso_p <- calc_metrics(
  B_true = individual_B,
  B_est = est_lasso_B_p,
  output_type = "mean"
  )

  metrics_varp <- data.frame(
  Method = "LASSO (Lag p)",
  Sensitivity = metrics_lasso_p$mean_sens,
  Specificity = metrics_lasso_p$mean_spec
  )

  ss_dat <- rbind(ss_dat, metrics_varp)
}

ss_dat
```

```
##                 Method Sensitivity Specificity
## 1        LASSO (Lag 1)       0.605   0.8296053
## 2 Multi-VAR (Standard)       0.845   0.7852632
## 3 Multi-VAR (Adaptive)       0.380   0.9850000
## 4        LASSO (Lag p)       0.585   0.8744737
```

```r
kable(ss_dat, digits = 3,
      caption = "Comparison of Sensitivity and Specificity")
```

Table 2: Comparison of Sensitivity and Specificity

| Method | Sensitivity | Specificity |
|---|---|---|
| LASSO (Lag 1) | 0.605 | 0.830 |
| Multi-VAR (Standard) | 0.845 | 0.785 |
| Multi-VAR (Adaptive) | 0.380 | 0.985 |
| LASSO (Lag p) | 0.585 | 0.874 |

## Sensitivity and Specificity (per subject)

```r
# LASSO
Smetrics_lasso <- calc_metrics(
  B_true,
```

```
  B_est = est_lasso_B,
  output_type = "subject"
  )

# multivar lasso
Smetrics_multivar_lasso <- calc_metrics(
  B_true,
  B_est = est_multivar_lasso_B,
  output_type = "subject"
  )

# multivar adaptive
Smetrics_multivar_adapt <- calc_metrics(
  B_true,
  B_est = est_multivar_adapt_B,
  output_type = "subject"
  )

# results
ss_dat_subject <- data.frame(
  Method = rep(c("LASSO (Lag 1)", "Multi-VAR (Standard)", "Multi-VAR (Adaptive)"), each = K),
  Sensitivity = c(Smetrics_lasso$sensitivity,
                  Smetrics_multivar_lasso$sensitivity,
                  Smetrics_multivar_adapt$sensitivity),
  Specificity = c(Smetrics_lasso$specificity,
                  Smetrics_multivar_lasso$specificity,
                  Smetrics_multivar_adapt$specificity)
  )

if (p != 1) {
  Smetrics_lasso_p <- calc_metrics(
  B_true = individual_B,
  B_est = est_lasso_B_p,
  output_type = "subject"
  )

  Smetrics_varp <- data.frame(
  Method = "LASSO (Lag p)",
  Sensitivity = Smetrics_lasso_p$sensitivity,
  Specificity = Smetrics_lasso_p$specificity
  )

  ss_dat_subject <- rbind(ss_dat_subject, Smetrics_varp)
}
```

```
desired_order <- c("LASSO (Lag 1)",
                   "LASSO (Lag p)",
                   "Multi-VAR (Standard)",
                   "Multi-VAR (Adaptive)")

summary_table <- ss_dat_subject %>%
  group_by(Method) %>%
  summarise(
    # --- Sensitivity ---
```

```
    Sens_Mean = mean(Sensitivity, na.rm = TRUE),
    Sens_SD   = sd(Sensitivity, na.rm = TRUE),

    # --- Specificity ---
    Spec_Mean = mean(Specificity, na.rm = TRUE),
    Spec_SD   = sd(Specificity, na.rm = TRUE)
  ) %>%

  mutate(
    `Sensitivity Mean (SD)` = sprintf("%.3f (%.3f)", Sens_Mean, Sens_SD),
    `Specificity Mean (SD)` = sprintf("%.3f (%.3f)", Spec_Mean, Spec_SD),
    Method = factor(Method, levels = desired_order)
  ) %>%

  arrange(Method) %>%

  select(Method, `Sensitivity Mean (SD)`, `Specificity Mean (SD)`)

summary_table
```

```
## # A tibble: 4 x 3
##   Method               `Sensitivity Mean (SD)` `Specificity Mean (SD)`
##   <fct>                <chr>                   <chr>
## 1 LASSO (Lag 1)        0.605 (0.093)           0.830 (0.050)
## 2 LASSO (Lag p)        0.585 (0.099)           0.874 (0.023)
## 3 Multi-VAR (Standard) 0.845 (0.083)           0.785 (0.021)
## 4 Multi-VAR (Adaptive) 0.380 (0.185)           0.985 (0.020)
```

```
kable(summary_table, caption = "Model Performance: Mean and Standard Deviation across Subjects")
```

Table 3: Model Performance: Mean and Standard Deviation across Subjects

| Method | Sensitivity Mean (SD) | Specificity Mean (SD) |
|---|---|---|
| LASSO (Lag 1) | 0.605 (0.093) | 0.830 (0.050) |
| LASSO (Lag p) | 0.585 (0.099) | 0.874 (0.023) |
| Multi-VAR (Standard) | 0.845 (0.083) | 0.785 (0.021) |
| Multi-VAR (Adaptive) | 0.380 (0.185) | 0.985 (0.020) |

## RMSFE (Root Mean Square Forecast Error)

```
compute_rmsfe_p <- function(B_est, test_data_matrix) {

  # B_est: Coefficient array for a single subject.
  #        Expected dimensions: [d, d, p] or [d, d] (if p=1)
  # test_data_matrix: The test data including the initial lags.
  #                   Structure: Rows 1 to p are the "Lags" (from Training end).
  #                              Rows (p+1) to End are the "Targets" (Test data).

  # Ensure B_est is a 3D array [d, d, p]
  # If B_est is a 2D matrix (implying p=1), convert it to 3D array [d, d, 1]
  if (is.matrix(B_est) || length(dim(B_est)) == 2) {
    d_dim <- nrow(B_est)
```

```r
    B_est <- array(B_est, dim = c(d_dim, d_dim, 1))
  }

  d <- dim(B_est)[1]      # Number of variables
  p <- dim(B_est)[3]      # Lag order
  n_total <- nrow(test_data_matrix) # Total rows in test set (including lags)

  # Define Actuals (Targets)
  # The actual values predicted start from row (p + 1)
  actuals <- test_data_matrix[(p + 1):n_total, ]

  # Initialize Prediction Container
  # Size matches the number of actual target observations
  preds <- matrix(0, nrow = nrow(actuals), ncol = ncol(actuals))

  # Rolling One-Step Ahead Forecast
  # Predict X_t based on X_{t-1}, ..., X_{t-p}
  # Loop t runs through the indices of the original test_data_matrix
  for (t in (p + 1):n_total) {

    # Initialize prediction vector for time t
    pred_t <- numeric(d)

    # Accumulate contributions from each lag: Sum(A_lag * X_{t-lag})
    for (lag in 1:p) {
      A_l <- B_est[, , lag]                 # Coefficient matrix for lag 'l'
      x_lag <- test_data_matrix[t - lag, ]  # Data vector at time t-l

      pred_t <- pred_t + (A_l %*% x_lag)
    }

    # Store prediction
    # Note: preds index starts at 1, corresponding to t-(p)
    preds[t - p, ] <- pred_t
  }

  # Calculate Root Mean Squared Forecast Error
  return(sqrt(mean((preds - actuals)^2)))
}



get_rmsfe_p <- function(B_est_obj, test_data_list, K) {

  # Initialize vector to store RMSFE for each subject
  rmsfe_vec <- numeric(K)

  for (k in 1:K) {

    # --- Logic to extract coefficients for Subject k ---

    # Case 1: Input is a 4D Array [d, d, p, K] (e.g., Multi-VAR output)
    if (is.array(B_est_obj) && length(dim(B_est_obj)) == 4) {
```

```
      B_k <- B_est_obj[, , , k]

    # Case 2: Input is a 3D Array [d, d, K] (e.g., Multi-VAR output when p=1)
    } else if (is.array(B_est_obj) && length(dim(B_est_obj)) == 3) {
      # Extract k-th slice and ensure it keeps array structure for compute_rmsfe_p
      B_temp <- B_est_obj[, , k]
      # Convert to [d, d, 1] so dimension checks pass
      B_k <- array(B_temp, dim = c(nrow(B_temp), ncol(B_temp), 1))

    # Case 3: Input is a List (e.g., LASSO output)
    } else if (is.list(B_est_obj)) {
      B_k <- B_est_obj[[k]]

    } else {
      # Case 4: Pooled/Shared model (Same matrix for everyone)
      B_k <- B_est_obj
    }

    # --- Compute RMSFE ---
    # Pass the extracted coefficients and the test data (which contains lags)
    rmsfe_vec[k] <- compute_rmsfe_p(B_k, test_data_list[[k]])
  }

  # Return the average RMSFE across all subjects
  return(rmsfe_vec)
}
```

```
# LASSO
rmsfe_lasso <- get_rmsfe_p(est_lasso_B, test_data, K)

# Multi-VAR (Standard)
rmsfe_multivar_lasso <- get_rmsfe_p(est_multivar_lasso_B, test_data, K)

# Multi-VAR (Adaptive)
rmsfe_multivar_adapt <- get_rmsfe_p(est_multivar_adapt_B, test_data, K)


rmsfe_dat <- data.frame(
  Method = rep(c("LASSO (Lag 1)", "Multi-VAR (Standard)", "Multi-VAR (Adaptive)"),each = K),
  RMSFE = c(rmsfe_lasso, rmsfe_multivar_lasso, rmsfe_multivar_adapt))


if (p != 1) {
  rmsfe_lasso_p <- get_rmsfe_p(est_lasso_B_p, test_data, K)

  rmsfe_dat_varp <- data.frame(
  Method = "LASSO (Lag p)",
  RMSFE = rmsfe_lasso_p
  )

  rmsfe_dat <- rbind(rmsfe_dat, rmsfe_dat_varp)
}
```

```r
# results
rmsfe_table <- rmsfe_dat %>%
  group_by(Method) %>%
  summarise(
    Mean = mean(RMSFE, na.rm = TRUE),
    SD   = sd(RMSFE, na.rm = TRUE),
  ) %>%

  mutate(
    `RMSFE Mean (SD)` = sprintf("%.3f (%.3f)", Mean, SD),
  ) %>%

  arrange(Method) %>%

  select(Method, `RMSFE Mean (SD)`)

rmsfe_table
```

```
## # A tibble: 4 x 2
##   Method                `RMSFE Mean (SD)`
##   <chr>                 <chr>
## 1 LASSO (Lag 1)         0.154 (0.042)
## 2 LASSO (Lag p)         0.141 (0.027)
## 3 Multi-VAR (Adaptive) 0.159 (0.047)
## 4 Multi-VAR (Standard) 0.146 (0.038)
```

```r
kable(rmsfe_table, caption = "Model Performance: Mean and Standard Deviation across Subjects")
```

Table 4: Model Performance: Mean and Standard Deviation across Subjects

| Method | RMSFE Mean (SD) |
|---|---|
| LASSO (Lag 1) | 0.154 (0.042) |
| LASSO (Lag p) | 0.141 (0.027) |
| Multi-VAR (Adaptive) | 0.159 (0.047) |
| Multi-VAR (Standard) | 0.146 (0.038) |

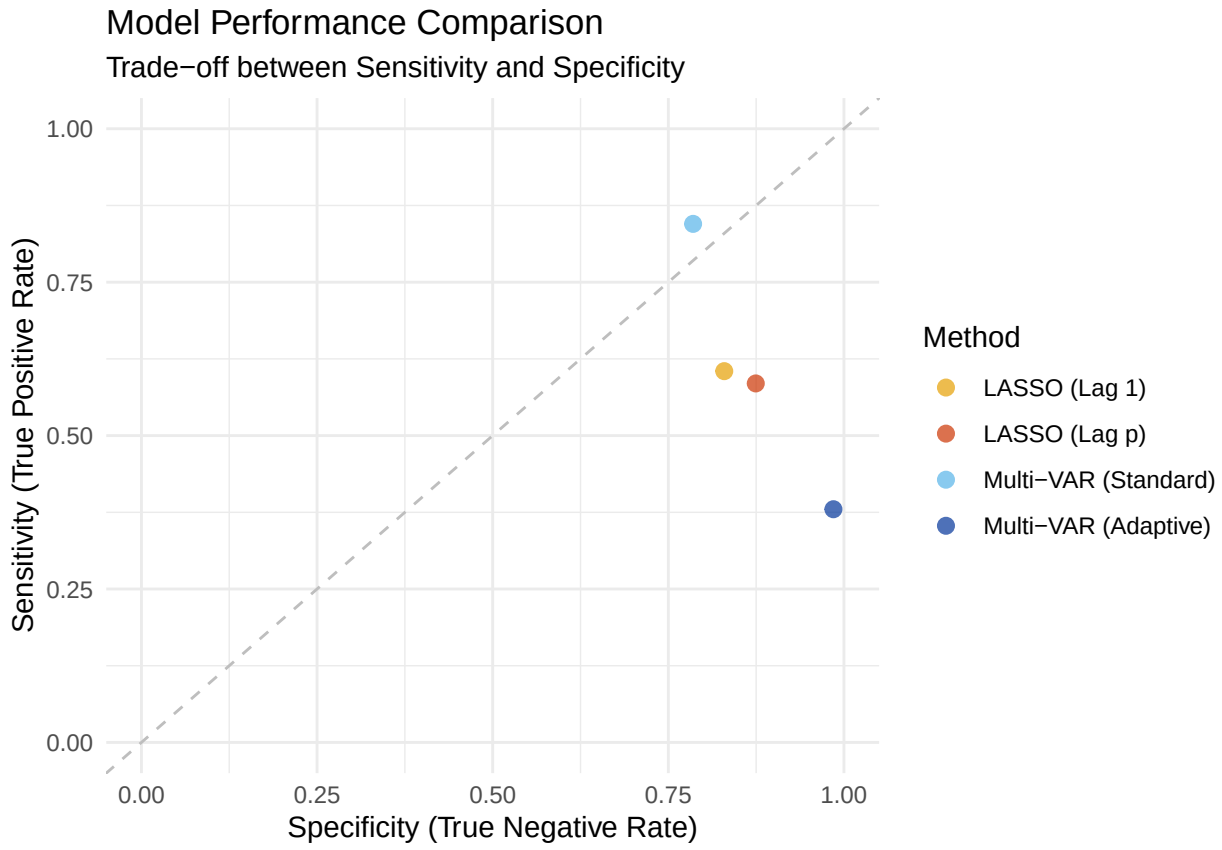
# Step 10: Visualization

## Sensitivity and Specificity (mean)

```r
ss_dat$Method <- factor(ss_dat$Method, levels = desired_order)
color_lab <- c("LASSO (Lag 1)" = "#E69F00",
               "LASSO (Lag p)" = "#CC3300",
               "Multi-VAR (Standard)" = "#56B4E9",
               "Multi-VAR (Adaptive)" = "#003399")

ggplot(ss_dat, aes(x = Specificity, y = Sensitivity, color = Method)) +
  geom_point(size = 2.5, alpha = 0.7) +
  xlim(0, 1) +
  ylim(0, 1) +
  geom_abline(intercept = 0, slope = 1, linetype = "dashed", color = "gray") +
```

```
  labs(
    title = "Model Performance Comparison",
    subtitle = "Trade-off between Sensitivity and Specificity",
    x = "Specificity (True Negative Rate)",
    y = "Sensitivity (True Positive Rate)"
  ) +
  theme_minimal() +
  scale_color_manual(values = color_lab)
```

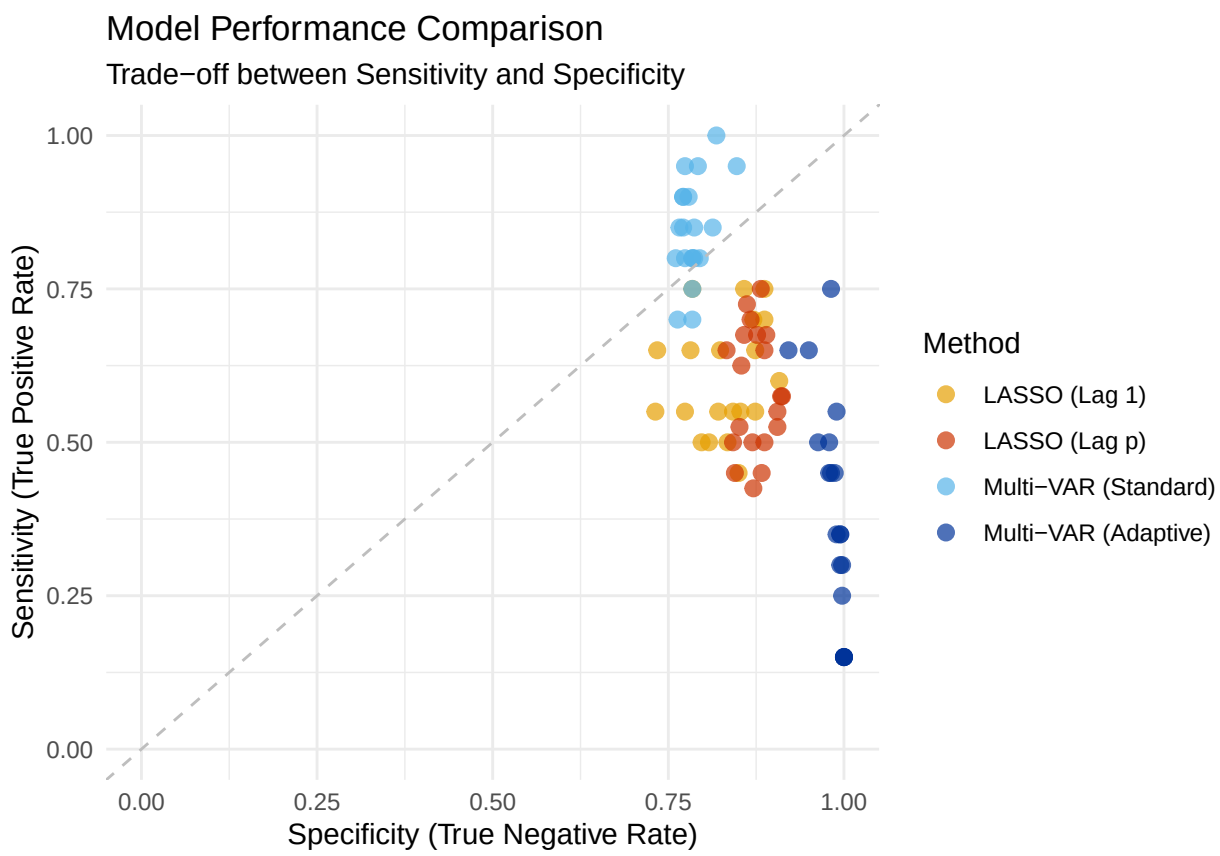



### Sensitivity and Specificity (per subject)

```
ss_dat_subject$Method <- factor(ss_dat_subject$Method, levels = desired_order)

# scatter plot
ggplot(ss_dat_subject, aes(x = Specificity, y = Sensitivity, color = Method)) +
  geom_point(size = 2.5, alpha = 0.7) +
  xlim(0, 1) +
  ylim(0, 1) +
  geom_abline(intercept = 0, slope = 1, linetype = "dashed", color = "gray") +
  labs(
    title = "Model Performance Comparison",
    subtitle = "Trade-off between Sensitivity and Specificity",
    x = "Specificity (True Negative Rate)",
    y = "Sensitivity (True Positive Rate)"
  ) +
```

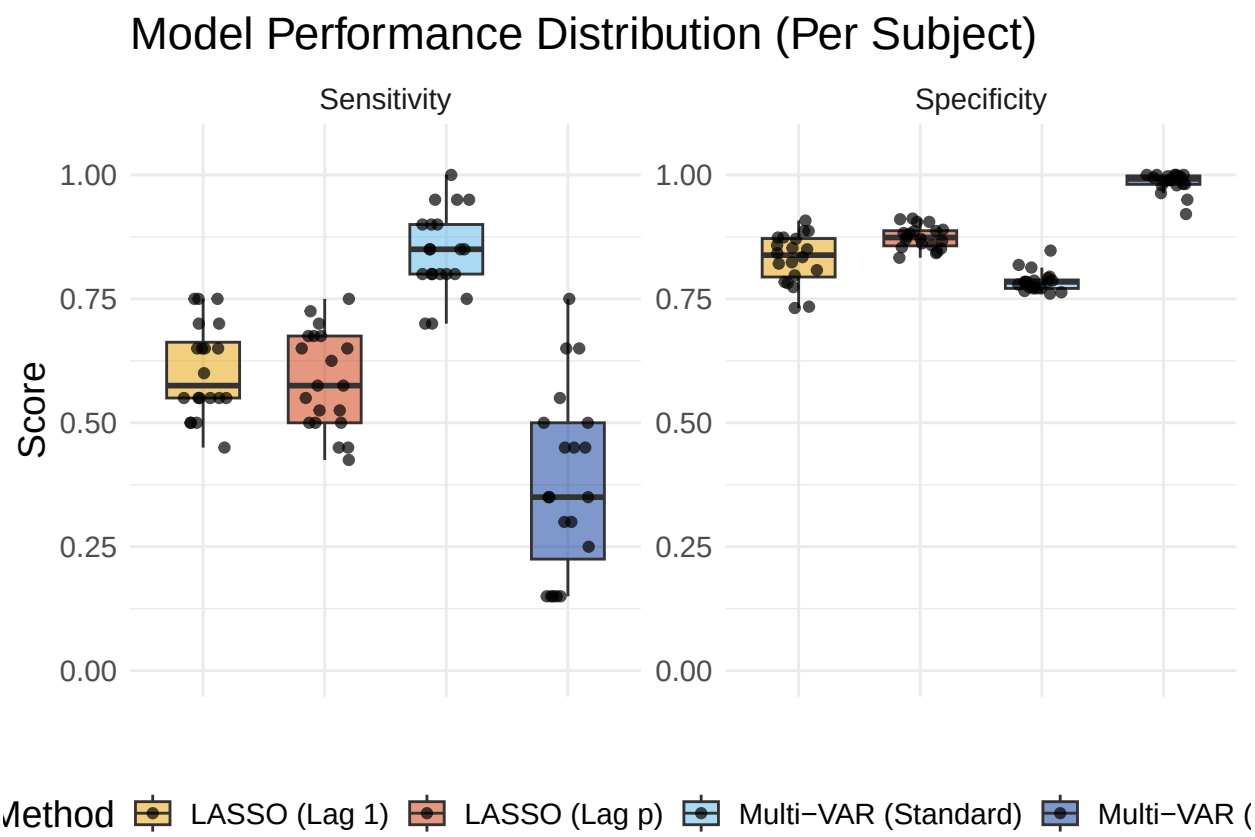
```
  theme_minimal() +
  scale_color_manual(values = color_lab)
```




```
# boxplot
ss_dat_subject_long <- melt(ss_dat_subject,
                        id.vars = c("Method"),
                        measure.vars = c("Sensitivity", "Specificity"),
                        variable.name = "Metric",
                        value.name = "Score")
ss_dat_subject_long$Method <- factor(ss_dat_subject_long$Method, levels = desired_order)

ggplot(ss_dat_subject_long, aes(x = Method, y = Score, fill = Method)) +
  geom_boxplot(alpha = 0.5, outlier.shape = NA, width = 0.6) +
  geom_jitter(width = 0.2, height = 0, size = 1.5, alpha = 0.7) +
  facet_wrap(~Metric, scales = "free_y") +
  scale_fill_manual(values = color_lab) +
  labs(
    title = "Model Performance Distribution (Per Subject)",
    #subtitle = "Boxplots show variability across 10 subjects",
    y = "Score",
    x = ""
  ) +
  ylim(0, 1.05) +
  theme_minimal() +
  theme(
    axis.text.x = element_blank(),
```

```
    axis.ticks.x = element_blank(),
    legend.position = "bottom",
    text = element_text(size = 14)
  )
```



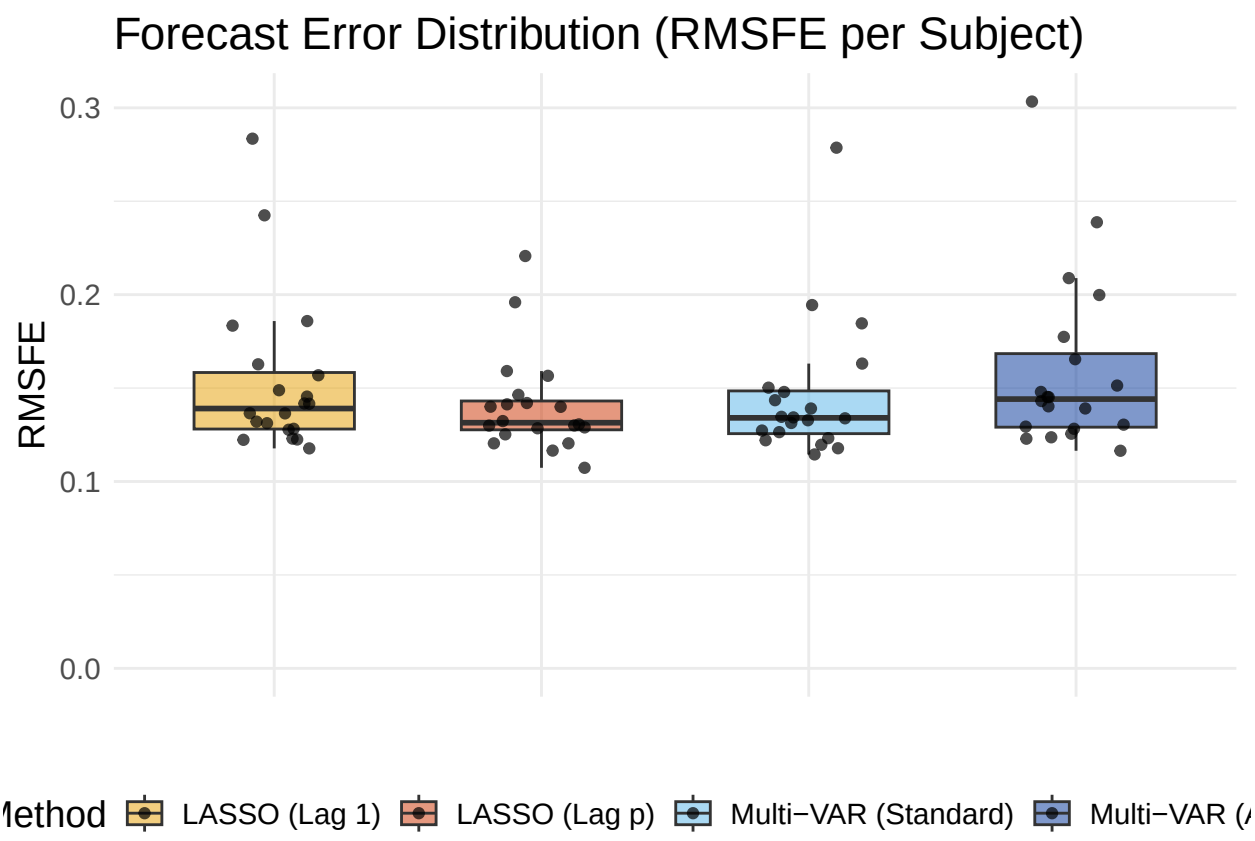

### RMSFE (per subject)

```
# boxplot
rmsfe_dat$Method <- factor(rmsfe_dat$Method, levels = desired_order)

ggplot(rmsfe_dat, aes(x = Method, y = RMSFE, fill = Method)) +
  geom_boxplot(alpha = 0.5, outlier.shape = NA, width = 0.6) +
  geom_jitter(width = 0.2, height = 0, size = 1.5, alpha = 0.7) +
  scale_fill_manual(values = color_lab) +
  expand_limits(y = 0) +

  labs(
    title = "Forecast Error Distribution (RMSFE per Subject)",
    #subtitle = "Lower is Better; Boxplots show variability across subjects",
    y = "RMSFE",
    x = ""
  ) +
  theme_minimal() +
  theme(
    axis.text.x = element_blank(),
```

```
    axis.ticks.x = element_blank(),
    legend.position = "bottom",
    text = element_text(size = 14)
  )
```




```
save.image(file = "setting12.RData")
```